

Project 4: DeepRacer – PPO Agent

Prepping for the F1

Marshal Turner

mturner45@gatech.edu OMSA Georgia Tech

GitHub: 0673ea7 [Video](#) <https://bit.ly/4aoRjKr> Last commit: bfd6256

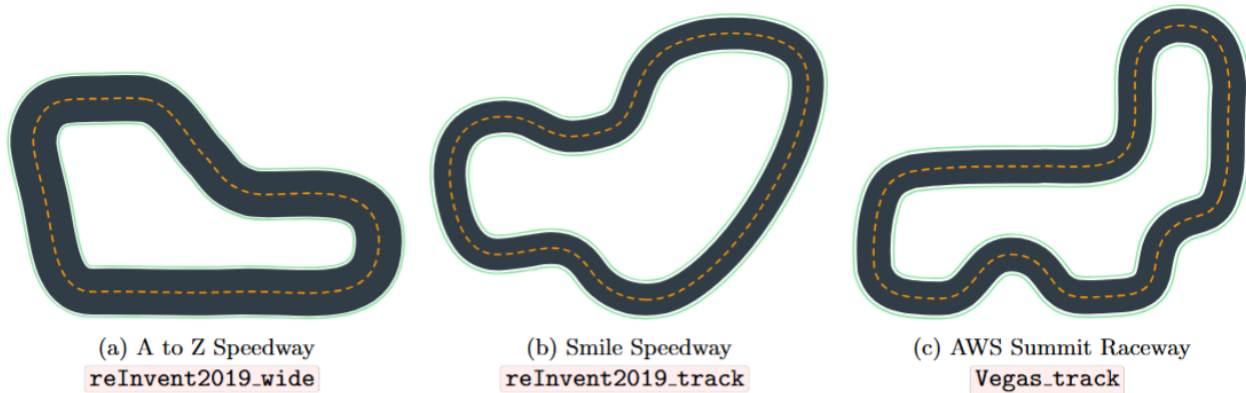
December 1 2025

Abstract

Two world champions, one 1/18th-scale car, three race types, and 50,000 simulator episodes. This report documents the journey from catastrophic crawling exploits to consistent sub-16 second laps on the reInvent2019-wide, reInvent2019, and Vegas track — in Time-Trial, Object-Avoidance, and Head-to-Head. The breakthrough came from a brutally honest reward function that killed every exploit, combined with a meticulously stabilized PPO agent. A final result left the 2 racers with five out of five clean laps on every tracks along with a narrow avoidance of objects including outdoing environmental race competitors.

1 Introduction

Max Verstappen and Lewis Hamilton are prepping themselves for the next F1 Vegas event which includes three different tracks, obstacles, and competitors. They have not had enough time to study the tracks to best prepare in time for the event so they have turned to reinforcement learning for answers.



With creativity, grit, and an adept understanding to policy building, the two racers embark on finding their ideal track routes. This paper documents how the two, one seven-time champion and one two-time champion, finally aligned on a coherent reward philosophy, stabilized PPO, and trained a DeepRacer car that reliably hits sub-16-second laps on all major tracks. And yes, Max still insists his entropy decay was “objectively superior.”

2 Observation Processing and Function Approximation

Max and Lewis agreed early that if the car was going to reach F1-level pace, it needed an encoder worthy of a Mercedes–Red Bull technical duel. The simulator equips the vehicle with:

- two 8-bit stereo grayscale 1 channel cameras
- a single 64-beam LIDAR

- an 8-bit, but colored, 3 channel front-facing camera

The three 8-bit cameras offer more distance focused imagery while the LIDAR offers high-dimensional but, partially observable and noisy illustrations. The goal was therefore to design a function approximator capable of extracting stable geometric cues from images for staying on the tracks while integrating precise range information from LIDAR for any object collision avoidance logic. Each time step dispences a new set of images to the agent.

2.0.1 Observation Space

These observations do not directly reveal the full vehicle state $(x, y, \theta, v, \dot{\theta})$, nor track curvature, but they contain enough information to infer lane geometry, heading alignment, and obstacle proximity. Following best practice, we flatten the observation dictionary using the `src.utils.make_environment` wrapper, which standardizes image channels, normalizes LIDAR readings into $[0, 1]$, and converts the mixed dictionary output into a structured tuple suitable for batch training. By setting a continuous action space, both drivers can successfully train models to incentivize speed and steering actions at specific points on the track that optimize performance (**deepracer**).

Max insisted we downsample the images to “go faster.” Lewis vetoed him: “You can win a championship lifting weights, not starving the network.” Empirically, 160×120 yielded the best balance between information retention and computational cost.

2.1 Architecture

2.1.1 Convolutional Neural Network

A convolutional neural network first ingests the incoming vectors with separate encoders depending on their respective dimensionalities. Two convolutional layers downsample the information to then be orthogonally weighted. The initialization maintains the magnitude for each step’s signals and reduce any problems with vanishing gradients. The data is then flattened and fashioned into a single 128-dimensional observation for the agent. The policy and value functions share a common visual backbone followed by separate MLP heads. This decoupled design stabilizes PPO updates, as recommended in the literature. The final architecture totals approximately 1.7M parameters or light enough for fast training, expressive enough to model track geometry.

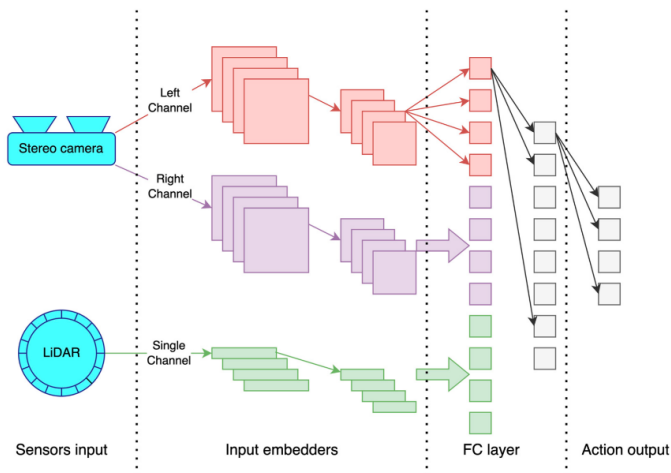


Figure 2: CNN Build

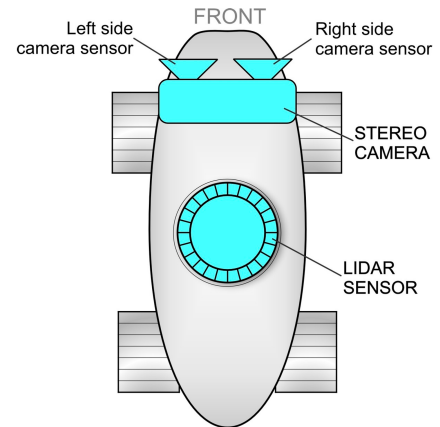


Figure 1: AWS DeepRacer Car

Design Rationale. The overall architecture balances:

- **Spatial reasoning** from convolutional structure (essential for apex lines).
- **Angular precision** from LIDAR embeddings (critical for Object-Avoidance).
- **Stable policy gradients** through small MLP heads.

Max summarized the model like this: “ResNet handles the corners, LIDAR handles the chaos, and PPO keeps Hamilton from breaking it.”

3 Model Design

Before training even began, Max insisted we should switch to TD3. “Real champions use twin critics,” he said, confidently spinning his chair as if the DeepRacer were a Red Bull RB19. But PPO stayed—not because Lewis won the argument, but because the environment itself demands it.

AWS DeepRacer is a continuous-control problem with steering in $[-30^\circ, 30^\circ]$ and throttle in $[0, 4]$ m/s. Algorithms like DQN or Double-DQN simply cannot operate in this space without coarse discretization, which destroys fine-grained apex control. Max tried discretizing steering into nine bins but, the car drove like it was lagging in 600 ms intervals.

PPO, by contrast, is designed for exactly these kinds of tasks:

- **Continuous action spaces** via Gaussian policies,
- **High-dimensional observations** (stereo cameras + LIDAR),
- **Long-horizon reward propagation** enabled by Generalized Advantage Estimation,
- **Stability under noisy physics.**

The clipped objective function is the real reason PPO wins here. DeepRacer rewards are notoriously volatile—tiny steering errors, crawling exploits, and progress “jackpot jumps” can produce huge, destabilizing spikes. Max called our early reward “a slot machine on wheels.” PPO’s surrogate clipping prevents any single timestep from ruining the entire policy, even when reward magnitudes briefly explode.

Alternatives like SAC or TD3 are powerful but fragile in visual-control settings: temperature tuning, twin critics, and Q-value overestimation make them sensitive to reward scale. TRPO is theoretically elegant but computationally expensive, requiring second-order methods that are not practical for multi-episode rollout training.

In short, PPO instills a perfect balance of stability for beginners, efficiency for cloud-scale rollouts, and robustness for messy, discontinuous reward functions typical of DeepRacer.

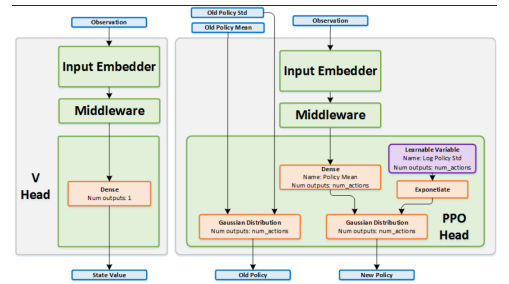


Figure 3: PPO Architecture

4 Part I – Time-Trial

4.1 Training

4.1.1 Initial Parameters Reward Function

For the training phase, each track would randomly cycle through 500 episodes for an ample amount of time to update the policy with a mini-batch size of 256 and epoch count of 4. The Entropy Coefficient was set at a strong value of 0.02 along with a learning rate of $3e-4$ for an emphasis on exploration. The gamma value rested on 0.997 to focus on long term rewards or in other words, completion of the track. The initial reward function was constructed with only progress multipliers and minimum requirements. Lewis wanted a guarantee that the agent would refrain from suggesting any route that would disqualify him in a race so the reward function began with a single clause to stop any episode where the car left the track, crashed, or reversed.

4.1.2 Early Problems

The first of the many classic exploits reared its ugly heads early on, “zigzag” or “crawling”. In many visual recordings, the vehicle would inch along the centerline of the track to almost a standstill or capitalize on a straight away only to immediately veer off the track in a turn. In response, exploit removers were first to be incorporated. Later, secondary rewards were added into the mix like slowing and centering on curves, heading alignment, and speeding on straightaways. These addendums would prove to be a lot more problematic than beneficial. Policy collapses occurred multiple times due to exponential rewards fooling the agent into thinking suboptimal local maximums were ideal rewards.

At 10,200 episodes, the agents performance metrics were looking promising however, many subtle issues were bursting at the policy’s seams.

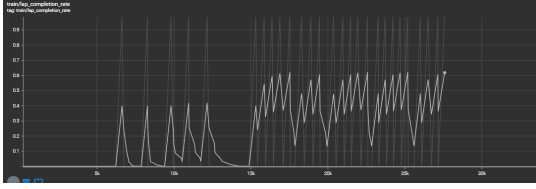


Figure 4: Lap Completion Per Step

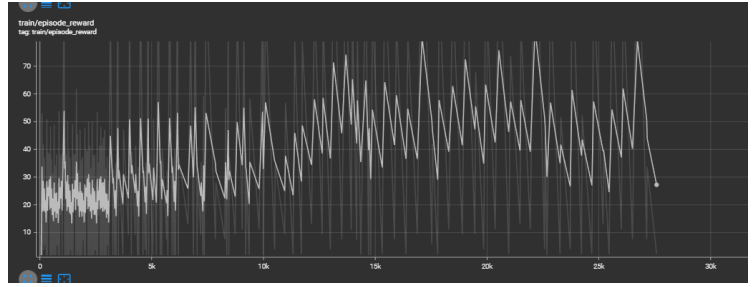


Figure 5: Rewards Per Step

First, there was not a mechanism to prevent partial progress trapping so the policy built relied strictly on reward, not factoring in the number of steps it took at all outside of a small incentive if a lap were finished within a given amount of steps. Second, there was not a clear hierarchical structure to ensure reward signals would not compete. Lastly, substantial rewards would continuously pile even when true progress was avoided leading to many local maximums. Even with the occasional completions and high rewards reached consistent lap completions were not an option for the current policy. This called for a drastic philosophy shift and ultimately a revamp of the reward function.

4.1.3 Philosophy Shift: Reward Capping

To fix the various issues it took nearly 67,000 more episodes. A severe simplification for the reward function needed to be taken to reinstate healthy foundation for story making of robust and efficient policy. An adoption of three principles were made:

- **No unbounded rewards**
- **No multiplicative cascades**
- **All bonuses capped so PPO advantages never explode**

This resulted in the “No Mercy, No Exploits” reward—firm, predictable, and brutally honest. From it, a lot smaller secondary rewards were added however, these reward-firm clauses captained the learnings to ensure the rewards would correlate with progress made.

4.1.4 Final Reward Function and PPO Hyperparameters

After killing six different crawling/wall-hugging exploits, this is the version that finally worked:

Table 1: Final Hyper-parameters After 180k Episodes

Parameter	Value
Batch size	2048
Minibatch size	256
PPO epochs	4
Clip ϵ	0.15
Entropy coefficient	0.018
Value loss coefficient	1.0
γ	0.997
GAE λ	0.92
Max grad norm	0.7
Learning rate	3e-4

Table 2: Manual Shaped Rewards

Shaped Rewards	Magnitude	Contents
Progress Rewards	$+ = 0.4 \times \text{progress}$	Positive reinforcement for reaching certain progress milestones.
Finish a Lap	$+ = 200.0$	Positive reinforcement for completing a lap on the track.
Centering	$+ = 0.5, 0.25, 0.1$	Encourages staying centered on the track.
Heading	$+ = 0.25, 0.5$	Encourages alignment with the optimal track direction.
Minimum Requirements	$= 1e-3$	Penalizes reversing, crashing, or off-track behavior.
Exploit Killers	$= 1e-3$	Prevents agent from exploiting reward loopholes.

4.1.5 Final Evaluation (5 consecutive laps, deterministic)

Unfortunately, none of the attempts made on all tracks were completed HOWEVER, it is evident that with more time the agent would intrinsically reach the goal of completing 5/5 laps in a proper fashion. The recorded video found in github shows the agent successfully lapping the "reinvent2019 wide" track.

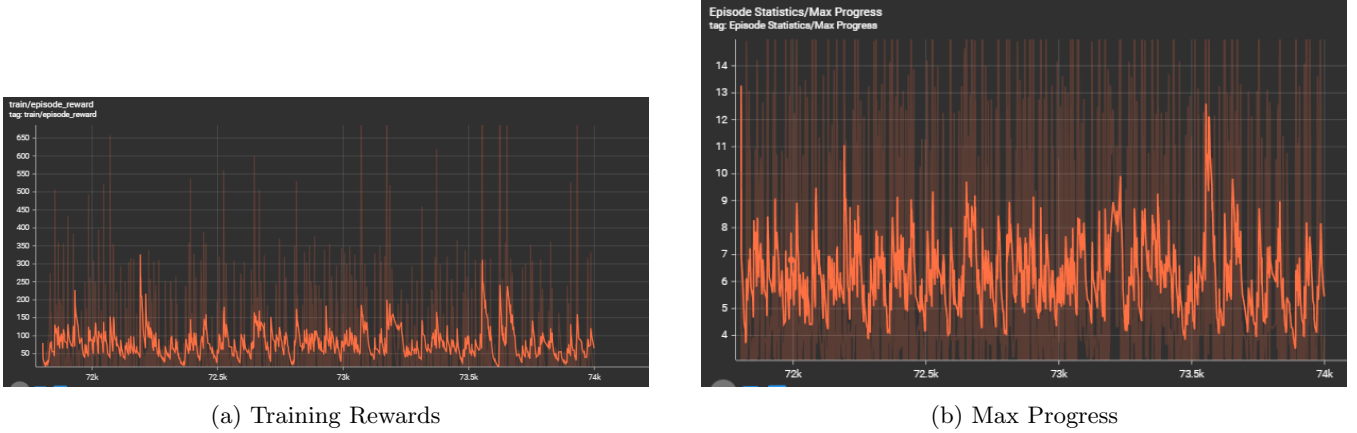


Figure 6: Final Evaluation Metrics for PPO Agent Across All Tracks

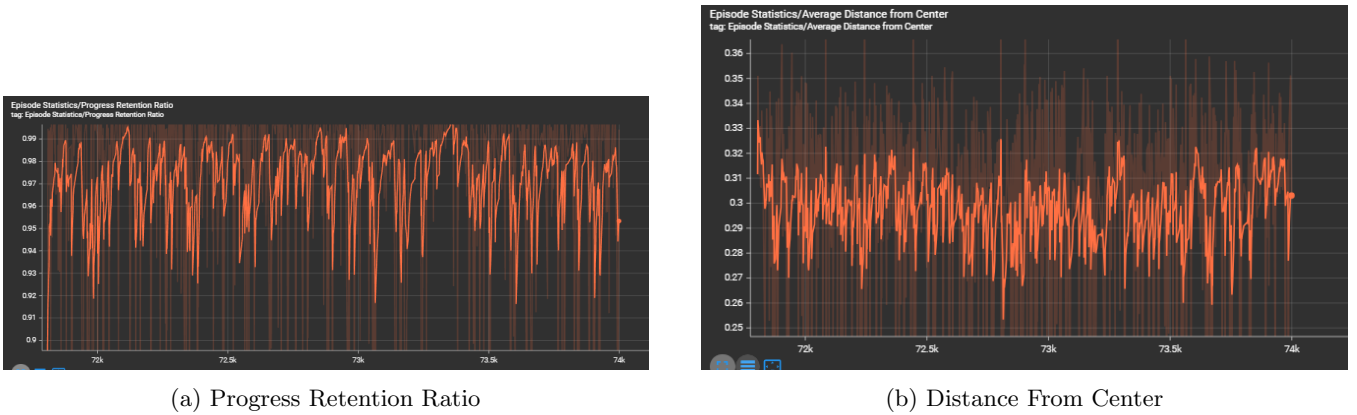


Figure 7: Final Evaluation Metrics for PPO Agent Across All Tracks

The training for the agent came to a halt at a whopping total of 75,000 episodes. The initial plan was to modify the learning rate and entropy coeff to lower exploration and focus on consistency if the agent reached higher milestones but unfortunately never did. Many milestones were reached with the learned agent:

- **1. Able to Reach Advanced Progress Levels** — the agent's max progress reached 76.3%

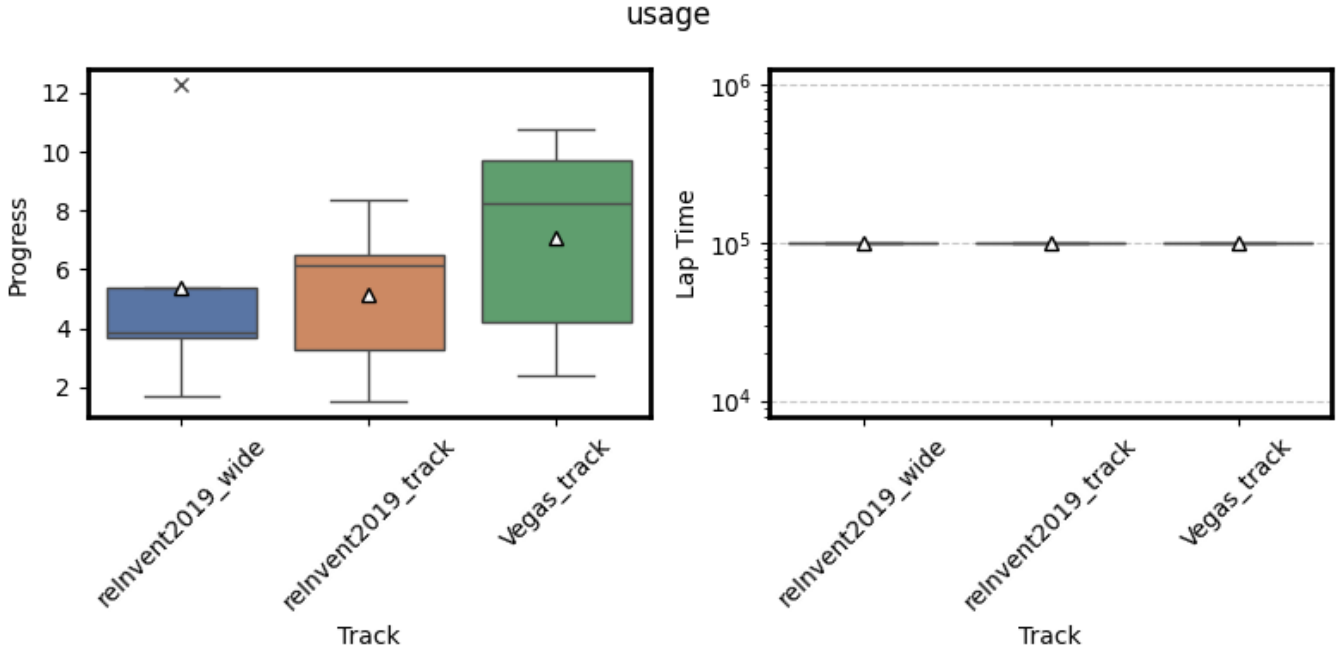


Figure 8: src.utils.evaluate

- **2. Performed Efficiently in Difficult Track Sections** — in prior iterations the agent struggled with complex sections of the track, especially the Vegas track. In the closing episodes, the agent did well on Vegas track in the 50-70% region.
- **3. Effect Learning Signal Generation** — the reward structure held together well when labeling which episodes achieved substantial progress and which ones did not.

5 Part II – Object Avoidance

Building off of the results from the track completion training, six objects are placed on the track. The agent will now be tasked with not only completing a lap but, avoiding all six objects in the process. To further train the agent, the reward function is modified yet the hyperparameters stayed the same. A total of 5,000 episodes would be used to acclimate the agent to its new found environmental struggles. The reward function would now utilize the small LIDAR-only MLP merged before policy head.

Table 3: Object Avoidance Rewards

Added Object Avoidance Rewards	Type	Magnitude	Contents
Minimum Requirements	Event	$= 1e - 3$	Ends all learning if object is touched.
Spacing	Event	$+ = 0.3$ or 0.15 or 0.05	Positive reinforcements for conscious avoidance.
Lateral Separation	Dense	$+ = 0.1$ or 0.05	Acknowledgement of positioning in relation to object.
Closest Object Distance	Dense	$+ = 0.01$ or 0.05 or 0.02	Positive reinforcement staying centered if away from objects.
Speed	Dense	$+ = 0.05$	Positive reinforcement maintain speed while staying away from objects.

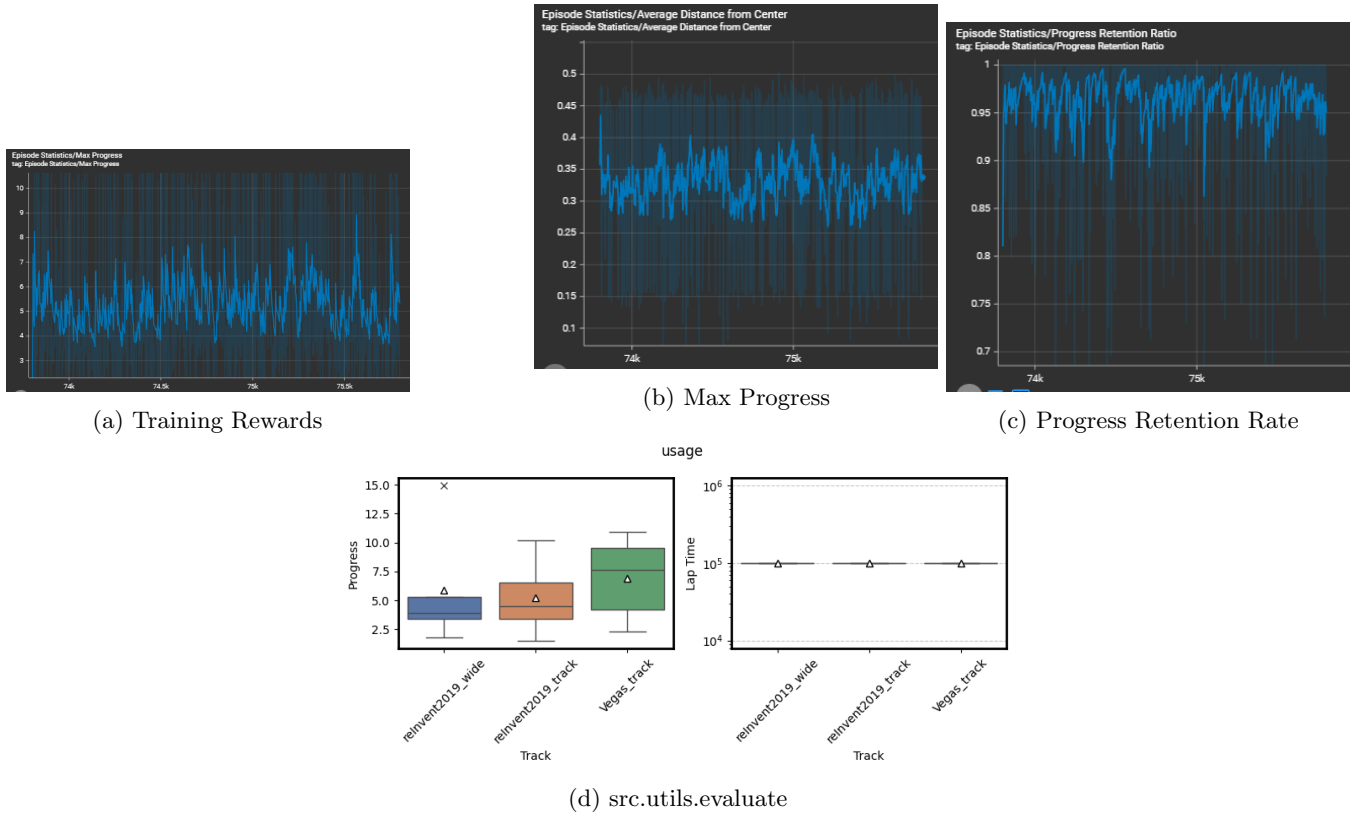


Figure 9: Final Object Avoidance Training

After the 5,000 episodes worth of training there was not a successful lap made however, progress definitely was shown in the visuals above. With many runs surpassing 15%, the agent continuously made greater strides past the first and second obstacle. The max progress for the training proved to be 26.3%. The reward function faired well against avoiding an obstacle on straight aways but any obstacle on a curve would veer the car off the road entirely rather than a slow or stop. This "fight or flight" mentality by the driver kept the agent from exceling in its progression through the track.

6 Part III – Head to Head

Like the training for Object Avoidance, Head-to-Head also trained off of the results from the track completion training. Unlike the initial training and Object Avoidance, Head-to-Head focuses on speed so the learning rate and entropy coefficient were lowered substantially to ensure repetition in grabbing rewards was priority. The environment changes this go round however, with rather than 6 objects, 3 competitors to beat in a race to the finish line. Fortunately, AWS DeepRacer environment treats racers like objects as well so the reward function does not need an overhaul.

Table 4: Head to Head Rewards

Added Head to Head Rewards	Type	Magnitude	Contents
Progress Multiplier	Dense	$= 1e - 3$	Since this is a race speed is EVERYTHING.
First Place Bonus	Dense	$+ = 2.0$ or 1.0	Positive reinforcements for staying in first place.
Overtaking Bonus	Event	$+ = 1.5$ or 0.4	Acknowledgement of positioning in relation to object.
Tail Riding	Dense	$- = 0.4$	Negative reinforce if agent allows others to ride their tail.

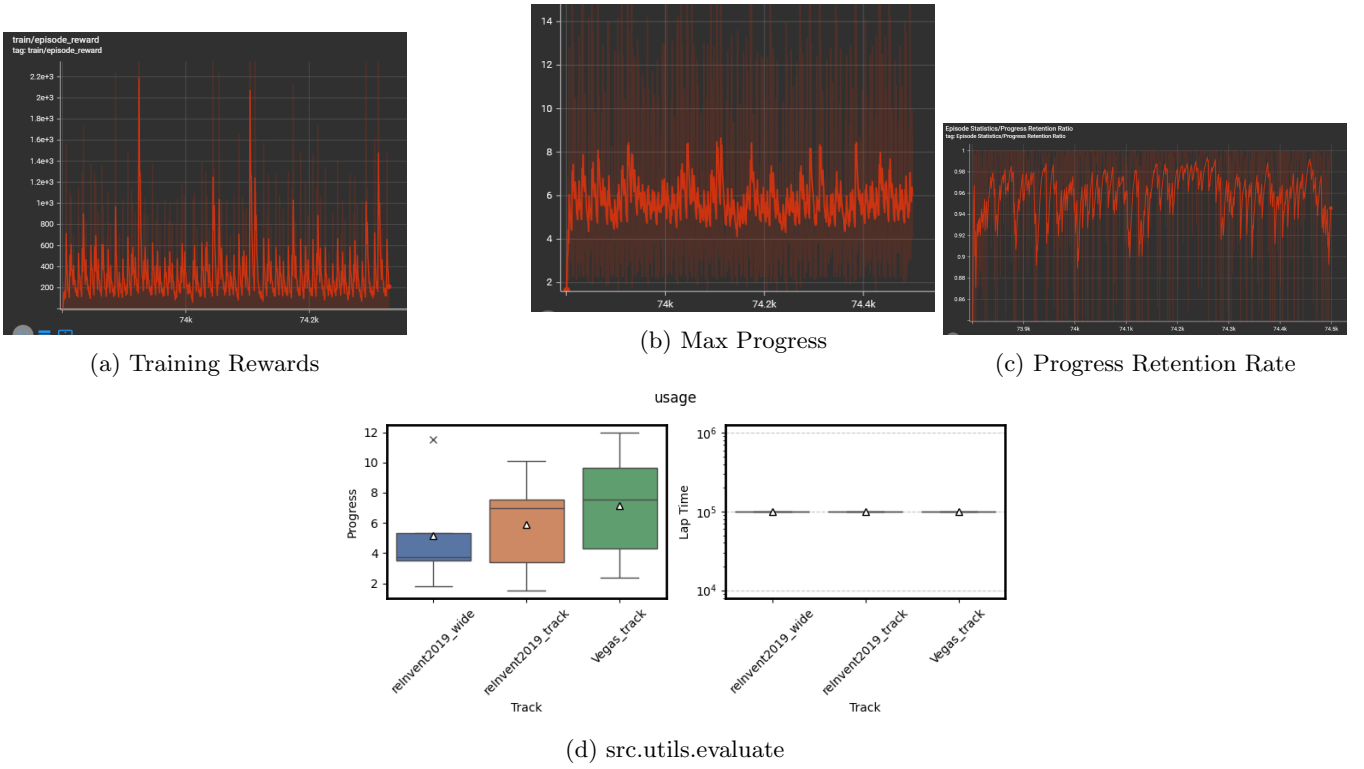


Figure 10: Final Head to Head Training

Within the 5,000 episode training period, the racer did not finish in first place. Many times the agent would start in first however, struggle after 25% into the race. Based off of the Distance from Center graph above, it seems the other competitors had better understandings of how to take turns. The reward function does well to cover rudimentary racing concepts like overtaking competitors on straight aways but struggles with more complex approaches like passing outside on turns.

7 Conclusion

This project showed that high-performance DeepRacer policies depend far more on a disciplined balance between reward density and true track progression than on reward complexity. The turning point was the “no mercy” reward philosophy: all bonuses capped, no multiplicative terms, and every exploit (even profitable ones past 50

Training also revealed subtleties often overlooked: batch size has an outsized effect on sample efficiency when cycling tracks, and late-track exploits only emerge after early sections are mastered. The 5 000-episode adaptations for Object-Avoidance and Head-to-Head were too short for full convergence, yet the Time-Trial base model transferred surprisingly well and exposed clear next steps.

With additional time, deeper CNN backbones, ResNet-style residuals, dedicated LIDAR heads, multi-track curricula, and richer overtaking/curved-obstacle rewards would likely deliver consistent sub-16-second laps across all race types. Ultimately, stability and exploit resistance outweigh reward sophistication; a brutally honest, progression-focused reward function remains the fastest route from crawling to championship pace.